SENTINEL-GNSS — Colab Training Notebook

Before running: Runtime ‣ Change runtime type ‣ **T4 GPU**

What lives where	
Code + all data CSVs	GitHub (cloned in Step 1)
Feature windows (.npz)	Built fresh each session from CSV
Checkpoints + figures	Google Drive (survives disconnects)

If Colab disconnects mid-training: Re-run Steps 1–3, then run the recovery cell at the bottom — it resumes from the last saved checkpoint automatically.

Step 1 — Clone repo from GitHub

All code and processed CSVs (including the labelled dataset) are pulled directly from GitHub.

```
import os; os.chdir('/content')
```

```
import os, shutil

GITHUB_REPO = 'https://github.com/Jorshuare/AI-Based-Prediction-for-GNSS-Signal-Degradation.git'
REPO_DIR    = '/content/sentinel-gnss'

# Always anchor to /content first so the kernel is never left in a deleted directory
%cd /content

if os.path.exists(f'{REPO_DIR}/.git'):
    print('Repo already cloned — pulling latest ...')
    %cd {REPO_DIR}
    !git pull
else:
    # Remove stale directory if it exists without .git (e.g. leftover from failed clone)
    if os.path.exists(REPO_DIR):
        print('Removing stale directory ...')
        shutil.rmtree(REPO_DIR)
    print('Cloning repo ...')
    !git clone {GITHUB_REPO} {REPO_DIR}
    %cd {REPO_DIR}

print(f'\nWorking directory: {os.getcwd()}')
!git log --oneline -5

# Confirm labelled CSV came down with the repo
csv_path = f'{REPO_DIR}/data/labelled/sentinel_gnss_labelled.csv'
assert os.path.exists(csv_path), f'CSV not found at {csv_path}'
import pandas as pd
df = pd.read_csv(csv_path)
print(f'\nDataset: {len(df):,} rows × {len(df.columns)} columns — ready.')
```

```
/content
Cloning repo ...
Cloning into '/content/sentinel-gnss'...
remote: Enumerating objects: 444, done.
remote: Counting objects: 100% (121/121), done.
remote: Compressing objects: 100% (97/97), done.
```

```
remote: Total 444 (delta 51), reused 86 (delta 23), pack-reused 323 (from 1)
Receiving objects: 100% (444/444), 270.30 MiB | 26.14 MiB/s, done.
Resolving deltas: 100% (130/130), done.
Updating files: 100% (243/243), done.
/content/sentinel-gnss

Working directory: /content/sentinel-gnss
4cddb00 (HEAD -> main, origin/main, origin/HEAD) fix: update model
c2d50f0 fix: balanced val subset for early stopping (500/class vs 97% CLEAN full val)
59c3212 fix: aux loss skipped for ablation models; baselines table handles flat metrics JSON
0aef1cd feat: aux t+0 head, warning weight [1,4,3], threshold tuning, drone cap, no-smote training
9531888 fix: train without smote"

Dataset: 97,393 rows × 41 columns — ready.
```

Step 2 — Install extra dependencies + verify GPU

Colab already ships with PyTorch, NumPy, pandas, scikit-learn, matplotlib, seaborn, scipy.
Only `imbalanced-learn` (SMOTE) needs installing.

```
!pip install -q imbalanced-learn

import torch
print(f'PyTorch  : {torch.__version__}')
print(f'CUDA     : {torch.cuda.is_available()}')
if torch.cuda.is_available():
    props = torch.cuda.get_device_properties(0)
    print(f'GPU      : {torch.cuda.get_device_name(0)}')
    print(f'VRAM     : {props.total_memory / 1e9:.1f} GB')
else:
    print('WARNING: No GPU — go to Runtime > Change runtime type > T4 GPU')
```

```
PyTorch  : 2.10.0+cpu
CUDA     : False
WARNING: No GPU — go to Runtime > Change runtime type > T4 GPU
```

Step 3 — Mount Google Drive (checkpoints + figures only)

Checkpoints are saved to Drive so training survives disconnects.
Data CSVs are **not** needed here — they came from GitHub in Step 1.

```
from google.colab import drive
import os, shutil

drive.mount('/content/drive')

REPO_DIR   = '/content/sentinel-gnss'
DRIVE_BASE = '/content/drive/MyDrive/sentinel-gnss'

# Folders that must persist across sessions
PAIRS = [
    (f'{REPO_DIR}/results/models/checkpoints', f'{DRIVE_BASE}/checkpoints'),
    (f'{REPO_DIR}/results/figures',            f'{DRIVE_BASE}/figures'),
]

for local, on_drive in PAIRS:
    os.makedirs(on_drive, exist_ok=True)
    os.makedirs(os.path.dirname(local), exist_ok=True)
```

```
        if os.path.isdir(local) and not os.path.islink(local):
            shutil.rmtree(local)   # remove empty local dir
        if not os.path.islink(local):
            os.symlink(on_drive, local)
        print(f'  {os.path.basename(local):15s} → {on_drive}')

    # Report any existing checkpoints (from a previous run)
    ckpt_dir = f'{DRIVE_BASE}/checkpoints'
    ckpts = sorted(f for f in os.listdir(ckpt_dir) if f.endswith('.pt'))
    if ckpts:
        print(f'\nExisting checkpoints on Drive ({len(ckpts)}):')
        for c in ckpts:
            print(f'  {c}')
        print('\nRun Step 5 with --resume to continue training.')
    else:
        print('\nNo existing checkpoints — will start fresh.')
```

```
Mounted at /content/drive
  checkpoints     → /content/drive/MyDrive/sentinel-gnss/checkpoints
  figures         → /content/drive/MyDrive/sentinel-gnss/figures

No existing checkpoints — will start fresh.
```

Step 4 — Build feature windows

Reads `data/labelled/sentinel_gnss_labelled.csv` → produces sliding-window tensors.
Skips automatically if windows already exist (safe to re-run).

```
%cd /content/sentinel-gnss
# Build SMOTE windows (used by baselines) — --force ensures y_0s label is included
!python -m src.models.feature_prep --force

# Build no-SMOTE windows (used by deep model — SMOTE creates incoherent temporal windows)
!python -m src.models.feature_prep --no_smote --force

# Sanity check
import numpy as np
print('\nWindow shapes (SMOTE — for baselines):')
for split in ('train', 'val', 'test'):
    d = np.load(f'data/processed/windows/{split}.npz')
    y0_label = "y_0s present" if "y_0s" in d else "y_0s MISSING"
    print(f'  {split:5s}  X={d["X"].shape}  CLEAN={np.sum(d["y_5s"]==0):,}  WARNING={np.sum(d["y_5s"]==1):,}  DEGRADED={np.sum(d["y_5s"]==2):,}  [{y0_label}]')

print('\nWindow shapes (no-SMOTE — for deep model):')
for split in ('train', 'val', 'test'):
    d = np.load(f'data/processed/windows_no_smote/{split}.npz')
    y0_label = "y_0s present" if "y_0s" in d else "y_0s MISSING"
    print(f'  {split:5s}  X={d["X"].shape}  CLEAN={np.sum(d["y_5s"]==0):,}  WARNING={np.sum(d["y_5s"]==1):,}  DEGRADED={np.sum(d["y_5s"]==2):,}  [{y0_label}]')
```

```
/content/sentinel-gnss
INFO  Loading /content/sentinel-gnss/data/labelled/sentinel_gnss_labelled.csv …
INFO    97,393 rows × 41 columns
INFO    label dist: {0: 73370, 1: 15075, 2: 8948}
INFO    split dist: {'train': 53077, 'val': 26525, 'test': 17791}
INFO    Split reassign: 'supervisor_drone_1'  val → train  (2,769 rows)
INFO    Split reassign: 'supervisor_drone_2'  val → train  (2,792 rows)
INFO    Split reassign: 'supervisor_drone_12'  val → train  (5,562 rows)
INFO  Feature columns (34): ['alt', 'baseline_sats', 'clock_bias', 'cnr_trend', 'cnr_variance', 'cycle_slips', 'dop_ratio', 'elevation_violations', 'fix_continuity', 'fix_transitions', 'gdop', 'hdop',
INFO    Scaler saved → /content/sentinel-gnss/data/processed/scaler.pkl
WARNING    Session 'scenario_a_r1' too short; skipping.
WARNING    Session 'scenario_a_r2' too short; skipping.
```

```
INFO    train: 63,703 windows  | 5s label dist: [46169 10600  6934]
INFO    val  : 12,896 windows  | 5s label dist: [12089   444   363]
INFO    test : 17,803 windows  | 5s label dist: [13740  3003  1060]
INFO  Applying SMOTE to training set …
INFO    SMOTE input  shape: (63703, 1020) | labels: [46169 10600  6934]
INFO    SMOTE output shape: (138507, 1020) | labels: [46169 46169 46169]
INFO    Saved train windows → /content/sentinel-gnss/data/processed/windows/train.npz  (138,507 samples)
INFO    Saved val   windows → /content/sentinel-gnss/data/processed/windows/val.npz  (12,896 samples)
INFO    Saved test  windows → /content/sentinel-gnss/data/processed/windows/test.npz  (17,803 samples)

=== Window summary ===
  train  X=(138507, 30, 34)  y_5s=[46169 46169 46169]
  val    X=(12896, 30, 34)  y_5s=[12089   444   363]
  test   X=(17803, 30, 34)  y_5s=[13740  3003  1060]
INFO  === --no_smote: saving class-weight-only windows to windows_no_smote/ ===
INFO  Loading /content/sentinel-gnss/data/labelled/sentinel_gnss_labelled.csv …
INFO    97,393 rows × 41 columns
INFO    label dist: {0: 73370, 1: 15075, 2: 8948}
INFO    split dist: {'train': 53077, 'val': 26525, 'test': 17791}
INFO    Split reassign: 'supervisor_drone_1'  val → train  (2,769 rows)
INFO    Split reassign: 'supervisor_drone_2'  val → train  (2,792 rows)
INFO    Split reassign: 'supervisor_drone_12'  val → train  (5,562 rows)
INFO  Feature columns (34): ['alt', 'baseline_sats', 'clock_bias', 'cnr_trend', 'cnr_variance', 'cycle_slips', 'dop_ratio', 'elevation_violations', 'fix_continuity', 'fix_transitions', 'gdop', 'hdop',
INFO    Scaler saved → /content/sentinel-gnss/data/processed/scaler.pkl
WARNING    Session 'scenario_a_r1' too short; skipping.
WARNING    Session 'scenario_a_r2' too short; skipping.
INFO    train: 63,703 windows  | 5s label dist: [46169 10600  6934]
INFO    val  : 12,896 windows  | 5s label dist: [12089   444   363]
INFO    test : 17,803 windows  | 5s label dist: [13740  3003  1060]
INFO    Saved train windows → /content/sentinel-gnss/data/processed/windows_no_smote/train.npz  (63,703 samples)
INFO    Saved val   windows → /content/sentinel-gnss/data/processed/windows_no_smote/val.npz  (12,896 samples)
INFO    Saved test  windows → /content/sentinel-gnss/data/processed/windows_no_smote/test.npz  (17,803 samples)

=== Window summary ===
  train  X=(63703, 30, 34)  y_5s=[46169 10600  6934]
  val    X=(12896, 30, 34)  y_5s=[12089   444   363]
  test   X=(17803, 30, 34)  y_5s=[13740  3003  1060]

Window shapes (SMOTE — for baselines):
  train  X=(138507, 30, 34)  CLEAN=46,169  WARNING=46,169  DEGRADED=46,169  [y_0s present]
  val    X=(12896, 30, 34)  CLEAN=12,089  WARNING=444  DEGRADED=363  [y_0s present]
  test   X=(17803, 30, 34)  CLEAN=13,740  WARNING=3,003  DEGRADED=1,060  [y_0s present]

Window shapes (no-SMOTE — for deep model):
  train  X=(63703, 30, 34)  CLEAN=46,169  WARNING=10,600  DEGRADED=6,934  [y_0s present]
```

```
!pip install -q xgboost
!python -m src.models.baselines
```

```
INFO   Loading windows …
INFO     Train: (138507, 1020)   Test: (17803, 1020)
INFO
── Tier 1: Majority-class ──────────────────────────────────
INFO
── Tier 2: C/N₀ threshold rule ────────────────────────────
INFO
── Tier 3: Random Forest ───────────────────────────────────
INFO     RandomForest  +5s  MacroF1=0.5444  MCC=0.4171
INFO     RandomForest  +15s  MacroF1=0.5328  MCC=0.4011
INFO     RandomForest  +30s  MacroF1=0.5225  MCC=0.3901
INFO
── Tier 3: XGBoost ─────────────────────────────────────────
INFO     XGBoost       +5s  MacroF1=0.5215  MCC=0.4232
INFO     XGBoost       +15s  MacroF1=0.5332  MCC=0.4117
INFO     XGBoost       +30s  MacroF1=0.5200  MCC=0.3908
INFO
```

```
INFO   ==============================================================================
INFO     Comparison Table — Test Set Macro-F1  (higher is better)
INFO   ==============================================================================
INFO     Method                      Justification                 +5s     +15s     +30s
INFO   ------------------------------------------------------------------------------
INFO     MajorityClass               Trivial floor              0.2904   0.0377   0.0380
INFO     CNR_Threshold               Domain rule (RTCM)         0.0375   0.0377   0.0380
INFO     RandomForest                Classical ML baseline      0.5444   0.5328   0.5225
INFO     XGBoost                     Classical ML baseline      0.5215   0.5332   0.5200
INFO   ==============================================================================
INFO     Macro-F1: equally weights CLEAN / WARNING / DEGRADED classes.
INFO     Random 3-class predictor would score ≈ 0.3333.
INFO   ==============================================================================
INFO
Full metrics saved → /content/sentinel-gnss/results/baselines/baseline_comparison.json
```

Step 5 — Train

`--resume` continues from the last Drive checkpoint if it exists, otherwise starts fresh.
Checkpoints are saved directly to Drive every 10 epochs + whenever a new best val F1 is reached.

```
import shutil, os, glob

# Wipe ALL checkpoints from Drive so previous runs don't interfere
drive_ckpt = '/content/drive/MyDrive/sentinel-gnss/checkpoints'
local_ckpt = '/content/sentinel-gnss/results/models/checkpoints'

for ckpt_path in [drive_ckpt, local_ckpt]:
    if os.path.exists(ckpt_path):
        for f in glob.glob(f'{ckpt_path}/*'):
            os.remove(f)
        print(f'Cleared: {ckpt_path}')

%cd /content/sentinel-gnss
# Deep model trained WITHOUT SMOTE.
# Class weights [1.0, 4.0, 3.0] handle imbalance (WARNING boosted most — lowest precision).
# Auxiliary t+0 head forces model to learn current state as prerequisite for prediction.
!python -m src.models.train --batch_size 256 --window_dir data/processed/windows_no_smote
```

```
Cleared: /content/drive/MyDrive/sentinel-gnss/checkpoints
Cleared: /content/sentinel-gnss/results/models/checkpoints
/content/sentinel-gnss
INFO  Device: cpu
INFO  Loading windows …
INFO    train batches: 249  val batches: 51  val_stop (balanced): 1089 samples
/content/sentinel-gnss/src/models/transformer_lstm.py:211: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
  self.transformer = nn.TransformerEncoder(
SentinelGNSS  |  parameters: 367,692
INFO
============================================================
INFO    Training SENTINEL-GNSS  |  367,692 parameters
INFO    Epochs: 0 → 150  |  Patience: 30
INFO  ============================================================

INFO  Epoch 001/150  | tr_loss=0.2419  val_loss=0.0916  | val_F1(5s/15s/30s): 0.824/0.797/0.761  | stop_F1=0.800  | lr=4.00e-04  219.7s
INFO    ★ New best stop macro-F1 = 0.7995  → checkpoint_best.pt
INFO  Epoch 002/150  | tr_loss=0.1468  val_loss=0.0867  | val_F1(5s/15s/30s): 0.788/0.751/0.732  | stop_F1=0.800  | lr=6.00e-04  220.0s
INFO    ★ New best stop macro-F1 = 0.7999  → checkpoint_best.pt
INFO  Epoch 003/150  | tr_loss=0.1281  val_loss=0.0785  | val_F1(5s/15s/30s): 0.809/0.782/0.784  | stop_F1=0.826  | lr=8.00e-04  232.6s
INFO    ★ New best stop macro-F1 = 0.8259  → checkpoint_best.pt
INFO  Epoch 004/150  | tr_loss=0.1172  val_loss=0.0889  | val_F1(5s/15s/30s): 0.777/0.790/0.767  | stop_F1=0.831  | lr=1.00e-03  223.1s
INFO    ★ New best stop macro-F1 = 0.8306  → checkpoint_best.pt
INFO  Epoch 005/150  | tr_loss=0.1100  val_loss=0.0798  | val_F1(5s/15s/30s): 0.839/0.816/0.798  | stop_F1=0.861  | lr=1.00e-03  222.9s
```

```
INFO    ★ New best stop macro-F1 = 0.8610  → checkpoint_best.pt
INFO  Epoch 006/150  | tr_loss=0.1021  val_loss=0.0838  | val_F1(5s/15s/30s): 0.844/0.746/0.685  | stop_F1=0.793  | lr=1.00e-03  229.3s
INFO  Epoch 007/150  | tr_loss=0.0991  val_loss=0.0739  | val_F1(5s/15s/30s): 0.826/0.803/0.768  | stop_F1=0.837  | lr=1.00e-03  221.2s
INFO  Epoch 008/150  | tr_loss=0.0946  val_loss=0.0828  | val_F1(5s/15s/30s): 0.818/0.783/0.768  | stop_F1=0.827  | lr=9.99e-04  221.6s
INFO  Epoch 009/150  | tr_loss=0.0907  val_loss=0.0876  | val_F1(5s/15s/30s): 0.819/0.760/0.708  | stop_F1=0.818  | lr=9.98e-04  228.7s
INFO  Epoch 010/150  | tr_loss=0.0887  val_loss=0.0935  | val_F1(5s/15s/30s): 0.788/0.747/0.750  | stop_F1=0.809  | lr=9.97e-04  220.5s
INFO    → Periodic checkpoint: checkpoint_epoch_010.pt
INFO  Epoch 011/150  | tr_loss=0.0850  val_loss=0.0931  | val_F1(5s/15s/30s): 0.785/0.742/0.739  | stop_F1=0.818  | lr=9.96e-04  228.8s
INFO  Epoch 012/150  | tr_loss=0.0843  val_loss=0.0956  | val_F1(5s/15s/30s): 0.805/0.772/0.762  | stop_F1=0.813  | lr=9.94e-04  220.5s
INFO  Epoch 013/150  | tr_loss=0.0813  val_loss=0.1071  | val_F1(5s/15s/30s): 0.774/0.764/0.753  | stop_F1=0.828  | lr=9.93e-04  220.4s
INFO  Epoch 014/150  | tr_loss=0.0768  val_loss=0.0972  | val_F1(5s/15s/30s): 0.821/0.792/0.718  | stop_F1=0.838  | lr=9.91e-04  225.5s
INFO  Epoch 015/150  | tr_loss=0.0769  val_loss=0.1177  | val_F1(5s/15s/30s): 0.788/0.695/0.692  | stop_F1=0.834  | lr=9.88e-04  223.0s
INFO  Epoch 016/150  | tr_loss=0.0729  val_loss=0.0977  | val_F1(5s/15s/30s): 0.823/0.770/0.722  | stop_F1=0.834  | lr=9.86e-04  220.7s
INFO  Epoch 017/150  | tr_loss=0.0702  val_loss=0.0965  | val_F1(5s/15s/30s): 0.820/0.778/0.738  | stop_F1=0.838  | lr=9.83e-04  229.5s
INFO  Epoch 018/150  | tr_loss=0.0678  val_loss=0.0932  | val_F1(5s/15s/30s): 0.815/0.790/0.749  | stop_F1=0.837  | lr=9.80e-04  221.5s
INFO  Epoch 019/150  | tr_loss=0.0679  val_loss=0.1105  | val_F1(5s/15s/30s): 0.835/0.738/0.698  | stop_F1=0.830  | lr=9.77e-04  225.0s
INFO  Epoch 020/150  | tr_loss=0.0682  val_loss=0.1014  | val_F1(5s/15s/30s): 0.835/0.770/0.742  | stop_F1=0.844  | lr=9.74e-04  225.7s
INFO    → Periodic checkpoint: checkpoint_epoch_020.pt
INFO  Epoch 021/150  | tr_loss=0.0626  val_loss=0.1038  | val_F1(5s/15s/30s): 0.801/0.762/0.718  | stop_F1=0.823  | lr=9.70e-04  220.1s
INFO  Epoch 022/150  | tr_loss=0.0623  val_loss=0.1008  | val_F1(5s/15s/30s): 0.831/0.794/0.751  | stop_F1=0.835  | lr=9.66e-04  220.4s
INFO  Epoch 023/150  | tr_loss=0.0603  val_loss=0.1027  | val_F1(5s/15s/30s): 0.805/0.765/0.718  | stop_F1=0.813  | lr=9.62e-04  221.0s
INFO  Epoch 024/150  | tr_loss=0.0590  val_loss=0.1104  | val_F1(5s/15s/30s): 0.817/0.769/0.709  | stop_F1=0.821  | lr=9.58e-04  219.5s
INFO  Epoch 025/150  | tr_loss=0.0582  val_loss=0.1086  | val_F1(5s/15s/30s): 0.799/0.756/0.694  | stop_F1=0.804  | lr=9.54e-04  220.2s
INFO  Epoch 026/150  | tr_loss=0.0557  val_loss=0.1038  | val_F1(5s/15s/30s): 0.828/0.751/0.716  | stop_F1=0.823  | lr=9.49e-04  220.1s
INFO  Epoch 027/150  | tr_loss=0.0558  val_loss=0.1034  | val_F1(5s/15s/30s): 0.816/0.796/0.729  | stop_F1=0.830  | lr=9.44e-04  221.0s
INFO  Epoch 028/150  | tr_loss=0.0540  val_loss=0.1116  | val_F1(5s/15s/30s): 0.807/0.781/0.654  | stop_F1=0.817  | lr=9.39e-04  220.8s
INFO  Epoch 029/150  | tr_loss=0.0540  val_loss=0.1104  | val_F1(5s/15s/30s): 0.816/0.780/0.666  | stop_F1=0.813  | lr=9.34e-04  220.1s
INFO  Epoch 030/150  | tr_loss=0.0522  val_loss=0.1093  | val_F1(5s/15s/30s): 0.828/0.803/0.720  | stop_F1=0.841  | lr=9.28e-04  220.6s
INFO    → Periodic checkpoint: checkpoint_epoch_030.pt
INFO  Epoch 031/150  | tr_loss=0.0495  val_loss=0.1061  | val_F1(5s/15s/30s): 0.817/0.792/0.701  | stop_F1=0.825  | lr=9.23e-04  221.7s
INFO  Epoch 032/150  | tr_loss=0.0485  val_loss=0.1072  | val_F1(5s/15s/30s): 0.822/0.776/0.692  | stop_F1=0.820  | lr=9.17e-04  222.4s
INFO  Epoch 033/150  | tr_loss=0.0474  val_loss=0.1148  | val_F1(5s/15s/30s): 0.810/0.752/0.688  | stop_F1=0.819  | lr=9.11e-04  223.4s
INFO  Epoch 034/150  | tr_loss=0.0475  val_loss=0.1179  | val_F1(5s/15s/30s): 0.807/0.744/0.718  | stop_F1=0.820  | lr=9.05e-04  219.8s
INFO  Epoch 035/150  | tr_loss=0.0463  val_loss=0.1079  | val_F1(5s/15s/30s): 0.817/0.782/0.708  | stop_F1=0.823  | lr=8.98e-04  223.8s
```

Step 6 — Evaluate

Loads `checkpoint_best.pt` from Drive, runs all 14 analyses, saves figures to Drive.

```
%cd /content/sentinel-gnss
# --tune_thresholds: sweeps WARNING/DEGRADED probability thresholds on val set
#   to maximise macro-F1. Free improvement — no retraining needed.
# --window_dir: val/test splits are identical to SMOTE dir, but keeps paths consistent.
!python -m src.models.evaluate --tune_thresholds --window_dir data/processed/windows_no_smote
```

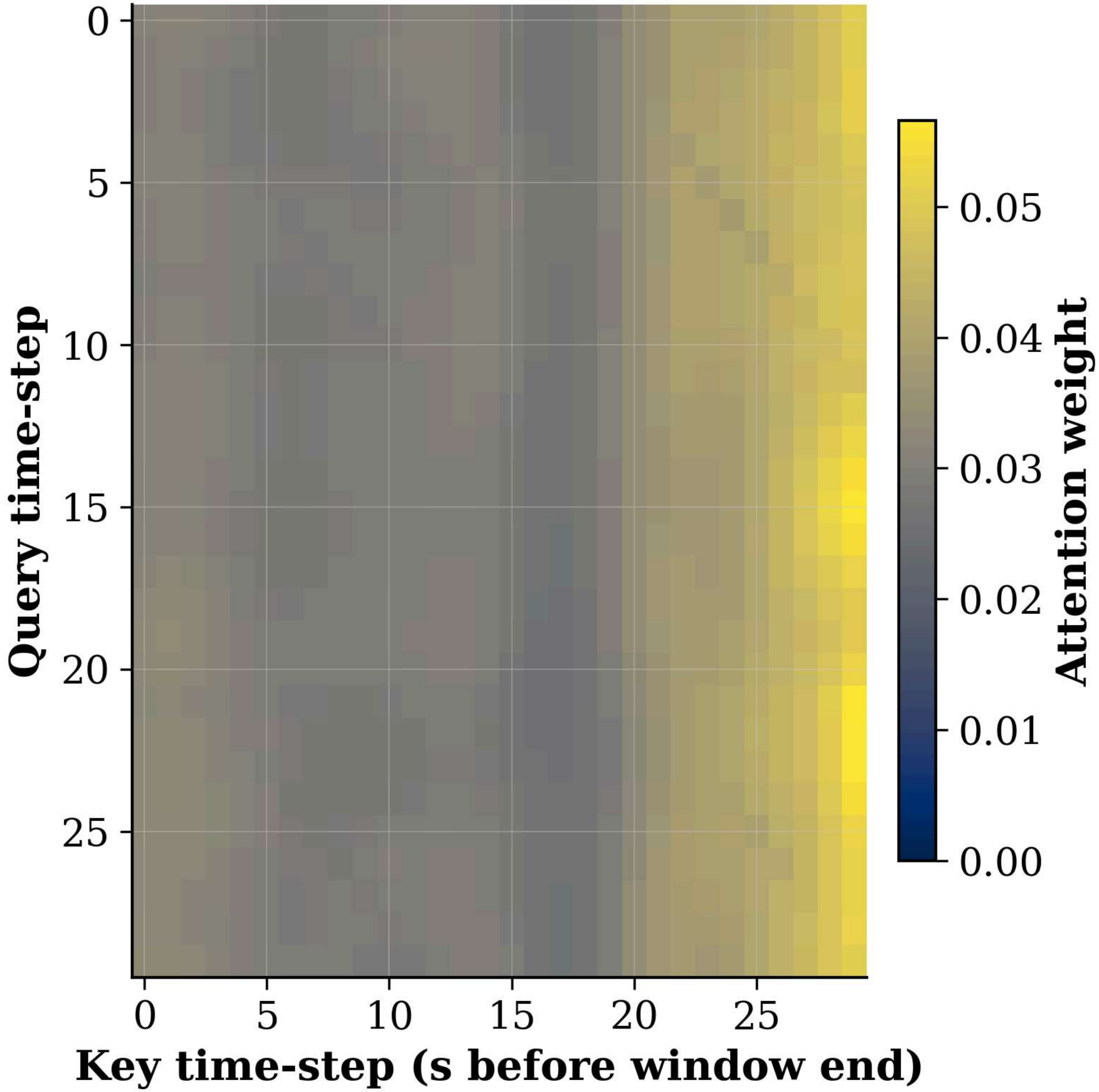
```
/content/sentinel-gnss
INFO  Device: cpu
/content/sentinel-gnss/src/models/transformer_lstm.py:211: UserWarning: enable_nested_tensor is True, but self.use_nested_tensor is False because encoder_layer.norm_first was True
  self.transformer = nn.TransformerEncoder(
SentinelGNSS  |  parameters: 367,692
INFO    Loaded model from checkpoint_best.pt  (epoch=5)
INFO    Running inference on 'test' split …
INFO    Tuning classification thresholds on val set …
INFO    Threshold +5s:  WARN=0.90  DEG=0.86  → val macro-F1=0.8496
INFO    Threshold +15s:  WARN=0.61  DEG=0.73  → val macro-F1=0.8296
INFO    Threshold +30s:  WARN=0.90  DEG=0.90  → val macro-F1=0.8094
INFO    Thresholds saved → /content/sentinel-gnss/results/figures/tuned_thresholds.json

  ── With tuned thresholds ──

─────────────────────────────────────────────────────────────────────
  SENTINEL-GNSS Evaluation Results  |  split = test+tuned
─────────────────────────────────────────────────────────────────────
Horizon      Acc   MacroF1     WtF1       κ       MCC
─────────────────────────────────────────────────────────────────────
```

```
  +5s     0.4905  0.4311    0.5810  0.2092  0.2821
  +15s    0.5053  0.4589    0.5896  0.2395  0.3191
  +30s    0.5146  0.4546    0.6013  0.2123  0.2727
────────────────────────────────────────────────────────────────────────

  +5s per-class breakdown:
    CLEAN       P=0.938  R=0.478  F1=0.633  n=13740
    WARNING     P=0.523  R=0.460  F1=0.489  n=3003
    DEGRADED    P=0.097  R=0.746  F1=0.172  n=1060

  +15s per-class breakdown:
    CLEAN       P=0.947  R=0.464  F1=0.623  n=13710
    WARNING     P=0.549  R=0.631  F1=0.587  n=3025
    DEGRADED    P=0.095  R=0.675  F1=0.166  n=1068

  +30s per-class breakdown:
    CLEAN       P=0.893  R=0.507  F1=0.646  n=13667
    WARNING     P=0.621  R=0.499  F1=0.553  n=3061
    DEGRADED    P=0.094  R=0.661  F1=0.164  n=1075

────────────────────────────────────────────────────────────────────────

INFO    Using tuned-threshold predictions for figures.

────────────────────────────────────────────────────────────────────────
  SENTINEL-GNSS Evaluation Results  |  split = test
────────────────────────────────────────────────────────────────────────
Horizon      Acc   MacroF1     WtF1       κ      MCC
────────────────────────────────────────────────────────────────────────
  +5s     0.4905  0.4311    0.5810  0.2092  0.2821
  +15s    0.5053  0.4589    0.5896  0.2395  0.3191
  +30s    0.5146  0.4546    0.6013  0.2123  0.2727
────────────────────────────────────────────────────────────────────────

  +5s per-class breakdown:
    CLEAN       P=0.938  R=0.478  F1=0.633  n=13740
    WARNING     P=0.523  R=0.460  F1=0.489  n=3003
    DEGRADED    P=0.097  R=0.746  F1=0.172  n=1060
```

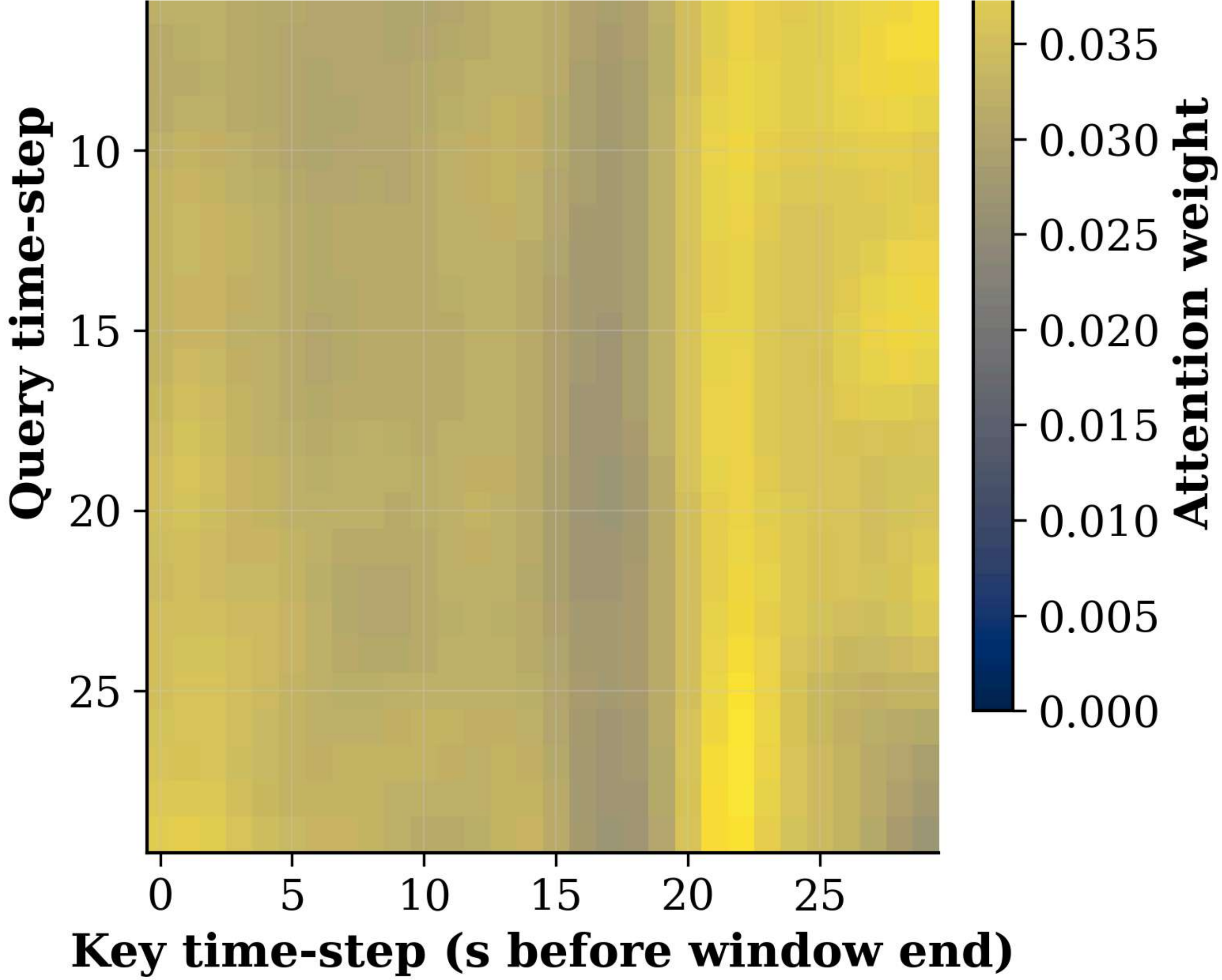
Step 7 — View all figures inline

```python
import glob
from IPython.display import Image, display

figs = sorted(glob.glob('/content/sentinel-gnss/results/figures/*.png'))
print(f'{len(figs)} figures saved to Drive:')
for f in figs:
    name = f.split('/')[-1]
    print(f'  {name}')
    display(Image(filename=f, width=950))
```

```
13 figures saved to Drive:
  attention_heatmap_clean.png
```




```
  attention_heatmap_degraded.png
```



attention_heatmap_warning.png